

Strategic Study Guide for the EEI4267 Requirement Engineering Final Examination

This study guide provides a comprehensive analysis of the EEI4267 Requirement Engineering final examination, based on a systematic review of examination papers from the 2019/20 academic year through the 2023/24 model answer paper. The focus is exclusively on preparing for a scenario-based examination format, where the application of theoretical knowledge to practical case studies is paramount.

Part 1: Deconstructing the EEI4267 Scenario-Based Examination

An analysis of past examinations reveals a highly consistent and predictable structure. Understanding this structure is the first step toward developing an effective preparation strategy. The examination is not merely a test of knowledge but a simulation of the tasks a Business Analyst (BA) performs.

1.1 The Anatomy of an EEI4267 Case Study

Every scenario-based section across the analyzed examination papers follows a distinct pattern.¹ This pattern consistently includes:

1. **A Narrative Scenario:** The examination presents a detailed narrative describing a business problem that requires a software solution. Examples include an e-ticket system for a cinema ¹, a biometric prisoner tracking system ¹, a mobile application to prevent train-elephant collisions ¹, an online event management platform ¹, and an online ticketing and merchandise system for a stadium.¹ This narrative provides the context, the business goals, and the key actors involved.
2. **A List of Preliminary Requirements:** Following the narrative, a numbered list of initial "requirements," "features," or "points" is provided. This list represents the raw, unrefined input a BA might receive early in a project. These lists are

intentionally imperfect; they often contain a mix of functional and non-functional requirements, ambiguities, conflicts, and poorly articulated ideas.¹ This list serves as the primary raw material for the majority of the questions that follow.

3. **A Series of Guided Questions:** The questions are structured to guide the student through the core activities of the Requirement Engineering (RE) process, using the case study and the provided list as a constant reference point. The student is essentially asked to perform the job of a BA on the given scenario.

1.2 The "Junior Business Analyst" Trope: A Pedagogical Insight

A recurring element, particularly in more recent papers, is the framing of the student's role as a senior professional guiding a less experienced colleague. The 2021/22 paper introduces "Nirash," a junior BA who needs guidance¹, and the 2022/23 paper features "Sally," a BA for whom the student must provide expert advice.¹

This is a deliberate pedagogical strategy. The questions are designed to test more than simple recall of definitions. For example, a question might be phrased as, "As a professional BA, you need to guide Nirash to successfully complete the requirement gathering".¹ This framing elevates the task from "What is requirement gathering?" to "How would you explain and apply requirement gathering techniques in a way a novice could understand and implement?"

This approach assesses a core competency of a senior analyst: the ability to communicate complex ideas clearly, justify decisions, and mentor team members. Therefore, the quality and clarity of the explanation are as crucial as the technical correctness of the answer. An effective response should adopt a guiding tone, structuring answers logically: start with a clear definition, provide a concrete example from the case study, and then explain the concept's practical importance within the project's context. This directly addresses the implicit demand of the question format.

1.3 The Inevitable Question Categories

Across all five years of examined papers, a clear set of recurring question categories emerges. Mastery of these topics is essential for success, as they form the backbone

of every exam.

- **Requirement Classification:** The most fundamental and consistently tested skill is the ability to differentiate between requirement types, particularly Functional and Non-Functional Requirements. This appears in every single analyzed paper.¹ Later papers also explicitly test the hierarchy of Business, User, and System requirements.¹
- **Requirement Elicitation and Gathering:** Every paper includes questions about the techniques used to discover and collect requirements from stakeholders.¹
- **Stakeholder Identification:** Students are frequently asked to identify the key stakeholders for the project described in the scenario.¹
- **UML Use Case Diagram:** A significant practical task in every paper involves modeling the system's functionality by drawing a main Use Case diagram, including actors, use cases, and relationships.¹
- **Methodology Comparison (Waterfall vs. Agile):** Questions requiring a comparison of the RE process in traditional plan-driven models versus modern agile frameworks are a common feature.¹
- **Requirement Management:** Topics such as the Requirement Traceability Matrix (RTM) and handling change requests are consistently tested, focusing on the ongoing management of requirements throughout the project lifecycle.¹

Part 2: Foundational Concepts: The Language of Requirements

A deep understanding of how to classify and describe requirements is the most critical foundational skill for the EEI4267 exam. The questions consistently demand not just definitions but the ability to identify, justify, and sometimes critique requirements from a given scenario.

2.1 Functional vs. Non-Functional Requirements (FR/NFR): The Core Distinction

This is the most frequently tested concept. Students are invariably asked to define both types and provide examples extracted from the case study's list of requirements.¹

- Functional Requirements (FRs): The "What"

A functional requirement specifies a function that a system or component must be able to perform. It describes a behavior, a piece of functionality, or an interaction with the system. FRs define what the system does.

- Example from ¹:

"Users must be able to browse for events by name or date or venue." This describes a specific function—the search capability—that the system must provide.

- Example from ¹:

"Jailors should be able to search prisoner details using prisoner ID and prisoner name." This defines a concrete action the system must support.

- Non-Functional Requirements (NFRs): The "How Well"

A non-functional requirement defines the quality attributes, constraints, and characteristics of the system. It specifies how the system should perform its functions. NFRs are often called the "-ilities" of a system (e.g., usability, reliability, performance, security).

- Example from ¹:

"The system should display availability of the tickets within five seconds." This does not describe a function but a performance constraint on a function.

- Example from ¹:

"The mobile application shall be developed with minimum storage requirements." This is a constraint on the system's resource usage.

- Example from ¹:

"The system must implement multi-factor authentication for user login..." This is a security requirement, a key type of NFR.

A notable pattern in the exam questions is the inclusion of poorly written or ambiguous requirements in the provided lists. This presents an opportunity to demonstrate a higher level of understanding. For instance, a requirement like "Railway drivers should easily understand the features available in the mobile application" ¹ is a weak, non-verifiable usability requirement. A basic answer would simply classify it as an NFR. However, a superior answer would also critique its quality. The 2022-23 paper directly tests this skill by asking, "Do you agree with Sally on the non-functional requirements he has written. If your answer is 'No,' please explain the reasons for your disagreement".¹ This indicates that the examiner expects students to assess the quality of requirements, not just classify them. When identifying NFRs, it is therefore advantageous to point out any ambiguity and suggest a more specific, measurable alternative (e.g., "A new driver must be able to perform a key task, such as acknowledging an alert, within 30 seconds of training"). This demonstrates a

proactive, analytical approach characteristic of an experienced BA.

2.2 The Requirement Hierarchy: Business, User, and System

More recent exams have focused on a more granular classification of requirements, asking students to differentiate between Business, User, and System requirements.¹ Understanding this hierarchy shows an appreciation for how high-level goals are translated into concrete software features.

- **Business Requirements (The "Why"):** These are the high-level objectives of the organization or customer. They state the business rationale for undertaking the project.
 - Example from ¹:
For the Arena Field ticketing system, a business requirement is "Improving Customer Experience" by reducing the hassle of waiting in queues. Another is "Increasing Revenue" through online ticket and merchandise sales.
- **User Requirements (The "What I Need to Do"):** These describe the goals or tasks that users must be able to perform with the system to achieve the business requirements. They are often expressed in natural language or as user stories.
 - Example from ¹:
"General Users need the ability to browse upcoming football matches, select seats, and purchase tickets online." This describes a task from the end-user's perspective.
- **System Requirements (The "How the System Enables It"):** These are detailed descriptions of the system's functions, services, and operational constraints. They specify what the system must do to fulfill the user requirements. This level is where functional and non-functional requirements are formally documented.
 - Example from ¹:
To fulfill the user requirement of secure purchasing, a system requirement is: "The system must provide a secure login with multi-factor authentication."
This is a detailed, verifiable specification for the development team.

2.3 Domain Requirements: The Context is Key

The 2023/24 model answer paper explicitly introduces the concept of domain requirements.¹ These are requirements that are derived from the application's specific domain rather than from the needs of a particular user. They often reflect business rules, industry standards, or physical laws relevant to the system's operational context.

- Example from ¹:
In the elephant tracking system, a domain requirement might be related to the physics of train braking distances or the biological patterns of elephant herd movements. A requirement like "Calculating the distance between animals and the train" ¹ is driven by the specific problem domain of railway safety.
- Example from ¹:
"Users under 18 should only have access to standard ticket options and limited merchandise, while users 18 and above can access premium seating and exclusive merchandise." This is not a universal software rule but a specific business rule from the domain of event ticketing and age-based access control.

Identifying domain requirements demonstrates an ability to think beyond generic software features and to analyze the specific real-world environment in which the system will operate. This is a hallmark of a mature and effective analyst.

Part 3: The Requirement Engineering Lifecycle in Practice

The examination questions are structured to test the student's ability to apply the core phases of the RE lifecycle to a practical scenario.

3.1 Phase 1: Elicitation & Gathering - Finding the Requirements

Questions about requirement gathering and elicitation techniques are a certainty on the exam.¹ Students are expected to list techniques, explain them, and, most importantly, select and justify the most appropriate techniques for the given scenario. The key is not just to name a technique but to explain

why it is suitable for the specific challenges and stakeholders presented in the case

study.

For instance, when asked to select and justify a technique, a robust answer follows a clear structure:

1. **State the Choice:** Clearly name the primary technique you are proposing.
2. **Define the Technique:** Briefly explain what the technique is and how it works.
3. **Justify its Relevance:** This is the most critical step. Connect the technique directly to the specifics of the case study. Why is this method better than others for this particular project? For example, for the "PrisTrack" system with jailors, administrators, and IT staff, a workshop is highly suitable because it facilitates consensus-building among diverse groups with potentially conflicting interests.¹
4. **Suggest a Secondary Technique:** Acknowledging that no single technique is perfect and suggesting a complementary one demonstrates a more nuanced understanding. For example, supplementing a workshop with one-on-one interviews with biometric experts would be a prudent strategy for the PrisTrack project.

The table below provides a quick-reference guide to the most commonly cited techniques.

Technique	Description	Best Used For	Pros	Cons
Interviews	One-on-one or small group discussions to gather in-depth information.	Understanding complex needs and individual perspectives from key stakeholders.	Rich, detailed data; allows for follow-up questions and clarification.	Time-consuming; susceptible to interviewer bias; may capture individual opinion over consensus.
Workshops	Structured, collaborative sessions with multiple stakeholders.	Achieving consensus, resolving conflicts, and prioritizing requirements collectively.	Fosters collaboration and shared ownership; can resolve issues quickly.	Can be difficult to schedule and manage; group dynamics may suppress some voices.
Surveys/Questionnaires	Written sets of questions distributed to a	Gathering quantitative data and	Efficient for large audiences; provides	Low response rate; no opportunity for

	large number of people.	opinions from a broad user base.	statistical data.	follow-up; questions can be misinterpreted.
Document Analysis	Reviewing existing business plans, process manuals, and system documentation.	Understanding the current "as-is" state and identifying existing business rules.	Provides context without taking up stakeholder time; based on existing knowledge.	Documentation may be outdated or inaccurate; may not reflect actual practice.
Prototyping	Creating an early, interactive model or mock-up of the system.	Clarifying requirements with users when they are unsure of their needs; validating design concepts.	Provides tangible feedback; helps users visualize the final product.	Can set unrealistic expectations; users may focus on cosmetic details.
KJ Method	A structured group process for organizing a large number of ideas into related groups (affinity diagramming).	Grouping and prioritizing ideas from a brainstorming session without duplication.	Democratic process; helps synthesize large amounts of qualitative data.	Requires a skilled facilitator; can be time-intensive.

3.2 Phase 2: Analysis & Refinement - Cleaning the Requirements

Once gathered, raw requirements must be analyzed for quality. The exam tests this by asking students to identify problems in the provided list, such as ambiguity or conflicts¹, or to rewrite a poor requirement into a better format like a User Story.¹

- **Identifying Ambiguity:** Ambiguous requirements use subjective or non-quantifiable terms. Look for words like "fast," "easy," "efficient," or "user-friendly." For example, "The system should be user-friendly" is ambiguous

because "user-friendly" means different things to different people.

- **Identifying Conflicts:** A conflict exists when two or more requirements are contradictory. For example, in the e-ticket system, the requirement "Users must be able to choose their seats" (implying a complex graphical interface) could potentially conflict with "The system should display availability of the tickets within five seconds" (a strict performance NFR).¹ A complex seat map might be slow to load, creating a trade-off that the BA must identify and resolve.
- **Writing User Stories:** This is a common Agile technique for capturing requirements from a user's perspective. The standard format is: As a <type of user>, I want <to perform some goal>, so that <I can achieve some benefit>. This format ensures the requirement is user-centric and linked to a business value.
 - Example based on ¹:
 - **Poor Requirement:** "Calculating the distance between animals and the train."
 - **Rewritten as a User Story:** "As a train driver, I want to receive a real-time visual and audible alert showing the distance to a detected elephant on the track, so that I can apply the emergency brakes in time to prevent a collision."

3.3 Phase 3: Documentation & Specification - Writing It Down

Proper documentation is crucial for communicating requirements to all project participants.

- **Software Requirements Specification (SRS):** The SRS is the official statement of what the system developers should implement. Questions about its purpose and desirable qualities are common.¹ A good SRS serves as the single source of truth for the project. The exam consistently emphasizes that a good SRS must be Complete, Consistent, Unambiguous, Verifiable, Traceable, and Modifiable. Conversely, "Ambiguous" is always presented as a property a good SRS must *not* have.¹
- **Requirement Traceability Matrix (RTM):** The RTM is a recurring topic, with questions asking for its definition, importance, and components.¹ An RTM is a document, typically a table, that traces the life of a requirement both forwards and backwards. It links each requirement to its source (e.g., stakeholder), its design specification, the code module that implements it, and the test case that verifies it. Its primary purposes are to ensure no requirements are missed and to

perform impact analysis when changes are requested. Key columns in an RTM typically include: Requirement ID, Description, Type, Source, Design Reference, Code Reference, Test Case ID, and Status.

3.4 Phase 4: Validation, Verification & Change Management

The final phase of the core RE process involves ensuring the requirements are correct and managing them as they evolve.

- **Validation vs. Verification:** The distinction between these two concepts was explicitly tested in the 2022-23 paper.¹
 - **Validation:** Answers the question, "Are we building the *right* product?" It is the process of checking that the requirements define the system that the customer really wants. Techniques include reviews, workshops, and prototyping.
 - **Verification:** Answers the question, "Are we building the product *right*?" It is the process of checking that the software meets its specification. Techniques include testing, inspections, and walkthroughs.
- **Handling a Change Request:** The 2020-21 paper presents a scenario where a client requests a change, but the team does not want to do it for free.¹ This tests the student's understanding of a formal change management process. A professional response involves these steps:
 1. **Log the Request:** Formally acknowledge and record the change request.
 2. **Perform Impact Analysis:** This is a critical step, highlighted in multiple exams.¹ Analyze the effect the proposed change will have on the project's scope, cost, schedule, resources, and other requirements.
 3. **Estimate and Present:** Provide the client with a formal estimate of the cost and timeline adjustments needed to implement the change.
 4. **Obtain Approval:** The client must formally approve the change along with the associated costs and schedule impacts. This directly addresses the "not for free" aspect of the scenario.
 5. **Update Project Baselines:** Once approved, update all relevant documentation, including the SRS and RTM, to reflect the change.

Part 4: Mastering the Practical Tasks: A Workshop on Use Case

Diagrams

The ability to create a main Use Case diagram is the single most important practical skill tested on the EEI4267 exam. It is a high-value question that appears in every single paper.¹

4.1 The Core Components

A Use Case diagram visualizes the interactions between external entities (actors) and the functions of the system (use cases).

Component	Visual	Description
System Boundary	A large rectangle	Represents the scope of the system being modeled. Everything inside the box is part of the system; everything outside is not.
Actor	A stick figure	An external entity that interacts with the system. It can be a human user (e.g., 'Customer') or another system (e.g., 'Payment Gateway'). Actors are always placed <i>outside</i> the system boundary.
Use Case	An oval	Represents a specific, user-visible functionality or goal that an actor can achieve with the system (e.g., 'Book Ticket'). Use cases are always placed <i>inside</i> the system boundary.
Association	A solid line	Connects an actor to a use case, indicating that the actor

		participates in that use case.
--	--	--------------------------------

4.2 The Relationships: Generalize, Include, Extend

Beyond simple associations, three key relationships are used to organize use cases.

- **Generalization:** A solid line with a hollow arrowhead, pointing from a specialized element to a more general one. It represents an "is-a-kind-of" relationship. It can be used for actors (e.g., a 'Premium User' is a kind of 'General User') or for use cases.
- **<<include>>:** A dashed arrow with the stereotype <<include>>, pointing from the base use case to the included one. This indicates that the behavior of the included use case is a mandatory part of the base use case. The base use case is incomplete without the included one. For example, 'Book Ticket' would <<include>> 'Process Payment'.
- **<<extend>>:** A dashed arrow with the stereotype <<extend>>, pointing from the extending use case to the base one. This indicates that the extending use case provides optional, additional functionality that may be executed under certain conditions. For example, 'Create Account' might be <<extended>> by 'Add Profile Picture'.

4.3 A Step-by-Step Construction Guide

1

Let's construct a main use case diagram for the "Online Ticketing and Merchandise System".¹

1. **Identify Actors:** The scenario explicitly mentions 'General Users (under 18)', 'Premium Users (18 and above)', and 'Administrators'. We can see a generalization relationship here: a 'Premium User' is a type of 'General User' but with additional privileges.
2. **Identify Major Goals (Use Cases):** From the scenario, we can identify goals like: 'Browse Matches', 'Book Ticket', 'Purchase Merchandise', 'Manage Match Schedules', 'Manage Ticket Prices', 'Monitor Sales'.

3. **Draw Boundary and Place Components:** Draw a large rectangle for the system boundary. Place the actors ('General User', 'Premium User', 'Administrator') outside the box. Place the use cases (ovals) inside.
4. **Connect Actors to Use Cases:**
 - The 'Administrator' connects to 'Manage Match Schedules', 'Manage Ticket Prices', and 'Monitor Sales'.
 - The 'General User' connects to 'Browse Matches' and 'Book Ticket'.
 - The 'Premium User' also connects to these, and additionally to 'Purchase Exclusive Merchandise'.
5. **Refine with Relationships:**
 - Draw a generalization arrow from 'Premium User' to 'General User' to show their relationship.
 - The 'Book Ticket' process logically requires selecting a seat and making a payment. Therefore, we can have 'Book Ticket' <<include>> 'Select Seat' and <<include>> 'Process Payment'.
 - Secure login is a prerequisite for most actions. A 'Login' use case could be included by many other use cases.

4.4 The Crucial Role of Assumptions

Every Use Case diagram question on the exam papers concludes with a prompt to "List down the assumptions you have made".¹ This is a mandatory part of the answer and is critical for demonstrating a professional understanding of modeling. Diagrams are simplifications, and stating your assumptions clarifies the scope and limitations of your model.

Examples of good assumptions for the ¹ scenario would be:

- "Assumption: The 'Process Payment' use case interacts with an external 'Payment Gateway' actor, which is not shown in this diagram for simplicity."
- "Assumption: User registration and account management are handled in a separate set of use cases not detailed here."
- "Assumption: The 'Administrator' actor represents all types of stadium management staff with system administration privileges."

The following checklist can be used to self-assess a Use Case diagram before finalizing the answer.

Checklist Item	Verification Question
System Boundary	Have I drawn a clear rectangle representing the system?
Actors	Are all external users/systems identified? Are they outside the boundary? Is generalization used where appropriate?
Use Cases	Does each oval represent a complete user goal? Is the name a verb-noun phrase (e.g., 'Book Ticket')?
Relationships	Have I used <<include>> for mandatory functionality and <<extend>> for optional functionality correctly?
Connections	Is every use case associated with at least one actor? Is every actor associated with at least one use case?
Assumptions	Have I written a clear list of the assumptions made to define the scope and simplify the diagram?

Part 5: The Methodological Context: Waterfall vs. Agile

The examination frequently includes questions that require a comparison of requirement engineering practices in traditional Waterfall models versus modern Agile frameworks like Scrum.¹

5.1 Core Philosophies

- **Waterfall (Plan-Driven):** This is a linear, sequential approach to software development. The RE phase is conducted comprehensively at the beginning of

the project to produce a detailed SRS. The goal is to define all requirements upfront and "freeze" them before design and development begin. Change is discouraged and managed through a formal, often cumbersome, change control process.

- **Agile (Change-Driven):** This is an iterative and incremental approach. RE is a continuous activity that occurs throughout the project. Requirements are expected to emerge and evolve. Instead of a single upfront SRS, requirements are often managed in a prioritized list called a Product Backlog. Change is welcomed and incorporated into future iterations (sprints).

5.2 Comparing RE Activities

While the fundamental goals of RE (understanding and documenting needs) exist in both models, the execution differs significantly. The following table provides a direct comparison of key aspects, which can be used to structure an answer to a "similarities and differences" question.

Aspect	Waterfall (Plan-Driven)	Agile (Scrum)
Requirement Stability	Requirements are assumed to be stable and are "frozen" after the initial analysis phase.	Requirements are expected to be volatile and are allowed to change throughout the project.
Documentation	Emphasizes a comprehensive, detailed Software Requirements Specification (SRS) document created upfront.	Emphasizes a "living" Product Backlog composed of User Stories. Documentation is "just enough" and evolves over time.
Change Management	Change is managed through a formal, rigorous Change Control Board (CCB) process. It is often seen as disruptive.	Change is embraced. New or changed requirements are added to the Product Backlog and prioritized for future sprints.
Stakeholder Involvement	Heavy involvement at the beginning (elicitation) and at	Continuous and active involvement of the Product

	major milestones (reviews). Less continuous interaction.	Owner and other stakeholders throughout the project (e.g., in sprint planning, reviews).
Key Artifacts	Business Requirements Document, Software Requirements Specification (SRS).	Product Backlog, Sprint Backlog, User Stories, Burndown Charts.
Process Timing	RE is a distinct, upfront phase that must be completed before the next phase (design) can begin.	RE is a continuous, ongoing activity that happens in every iteration (sprint).

Similarities: Despite these differences, both approaches share fundamental goals. Both must identify stakeholders, understand the core business problem, elicit needs, and document requirements in some form to guide development and testing. The necessity of RE itself does not disappear in Agile; only its timing, formality, and artifacts change.

5.3 Agile-Specific Concepts

- **Agile Inception:** The 2023-24 paper introduces the concept of an Agile Inception (or Sprint 0).¹ This is not a full-blown upfront analysis phase like in Waterfall. Instead, it is a short, time-boxed initial phase where the team and stakeholders align on the project vision, high-level scope, initial architecture, and a preliminary Product Backlog. The goal is to do "just enough" planning to get the project started on a solid footing.
- **Online Tools:** The exams also acknowledge the practical reality of modern development by referencing online tools for managing requirements in an Agile context, such as Jira or Trello.¹ Mentioning these tools demonstrates practical, industry-relevant knowledge.

Part 6: A Strategic Framework for Answering Scenario-Based Questions

Success in the EEI4267 final exam depends not only on knowledge but also on a strategic approach to reading, analyzing, and answering the questions under time pressure.

6.1 The First 15 Minutes: Active Reading and Annotation

Use the initial part of the examination to deconstruct the case study methodically. Do not start writing answers immediately.

1. **First Pass (Global Understanding):** Read the entire case study and all its associated questions once through. This gives a holistic view of the problem and the specific tasks that will be required.
2. **Second Pass (Active Annotation):** Read the case study narrative and the list of requirements again, this time with multiple highlighters or annotation methods.
 - **Identify Actors and Stakeholders:** Mark every person, group, or external system mentioned (e.g., 'Admin user', 'Users', 'jailors', 'Transport ministry', 'Premium Users').
 - **Identify Potential Functional Requirements:** Mark all action-oriented statements that describe what the system must *do* (e.g., "insert / delete data", "browse for events", "calculate the distance").
 - **Identify Potential Non-Functional Requirements:** Mark all statements describing qualities or constraints (e.g., "within five seconds", "24/7", "maximum of six concurrent users", "securely").
 - **Identify Potential Domain Requirements:** Mark any rules specific to the business context (e.g., rules about ticket sales, prisoner tracking protocols, age-based access).
3. **Annotate the List:** As you read the provided list of requirements, make brief notes in the margin. Label them ("FR," "NFR," "Ambiguous," "Conflict with #X") to pre-process the information for later questions.

This active reading process transforms you from a passive reader into an active analyst, preparing the raw material for your answers before you even begin writing them.

6.2 Structuring Your Answers for Maximum Marks

Clarity, structure, and directness are key to scoring well.

- **Follow the Question's Structure:** If a question asks for "three techniques" or "two differences," use a numbered or bulleted list. This makes your answer easy for the examiner to read and grade.
- **Use the Define, Explain, Example (DEE) Method:** For any question that asks you to explain a concept, this is a powerful and reliable structure.
 1. **Define:** Provide a concise, accurate definition of the term.
 2. **Explain:** Explain its purpose and importance in the context of requirement engineering.
 3. **Example:** Provide a specific, relevant example drawn directly from the case study.
- **Adopt the "Senior BA" Persona:** As discussed in Part 1, frame your explanations as if you are guiding a junior colleague. This will naturally lead to answers that are clearer, more logical, and better structured.
- **Manage Your Time:** Allocate your time based on the marks assigned to each question. A 12- or 15-mark Use Case diagram deserves significantly more time and attention than a 4-mark definition question. Plan your time before you start writing and stick to your plan as closely as possible.

Works cited

1. FinalExaminationPaper_EEI4267_2023_24_ModelAnswer.pdf